History Of GPU Computing

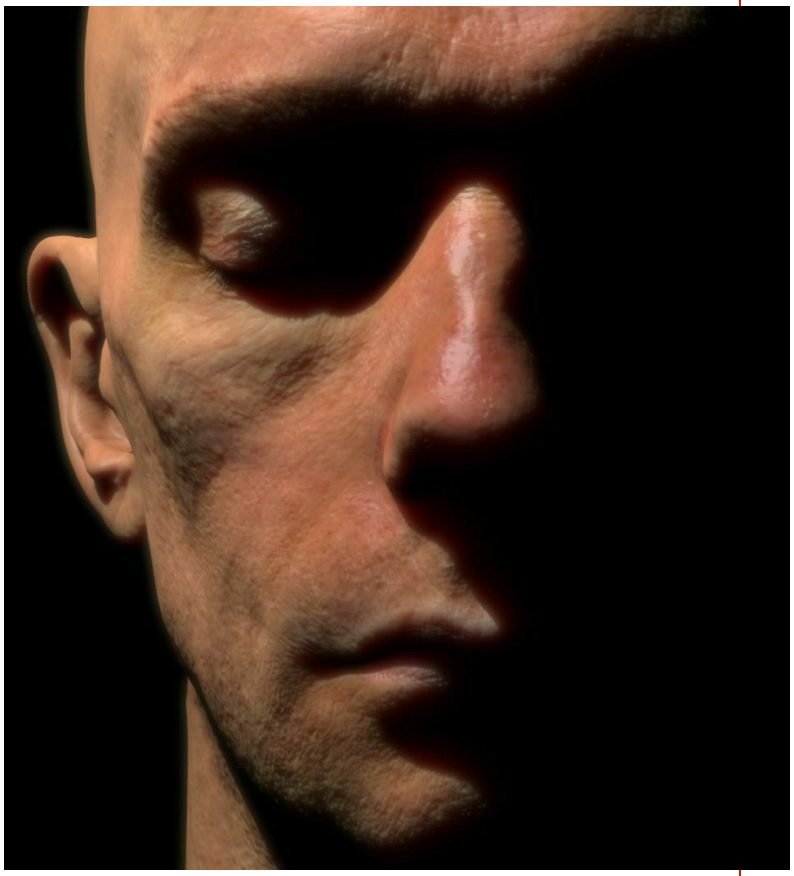
Computer Science Department
Eastern Washington University
Yun Tian (Tony) Ph.D.

Outline Today

- Overview of Graphics
- Graphics Pipeline and its Evolvement
- Programmable Array of Unified GPUs processors.
- Auto Scalable GPUs.

Graphics Overview

- Make great images
  - intricate shapes
  - complex optical effects
  - seamless motion
- Make them fast
  - invent clever techniques
  - use every trick imaginable
  - build monster hardware



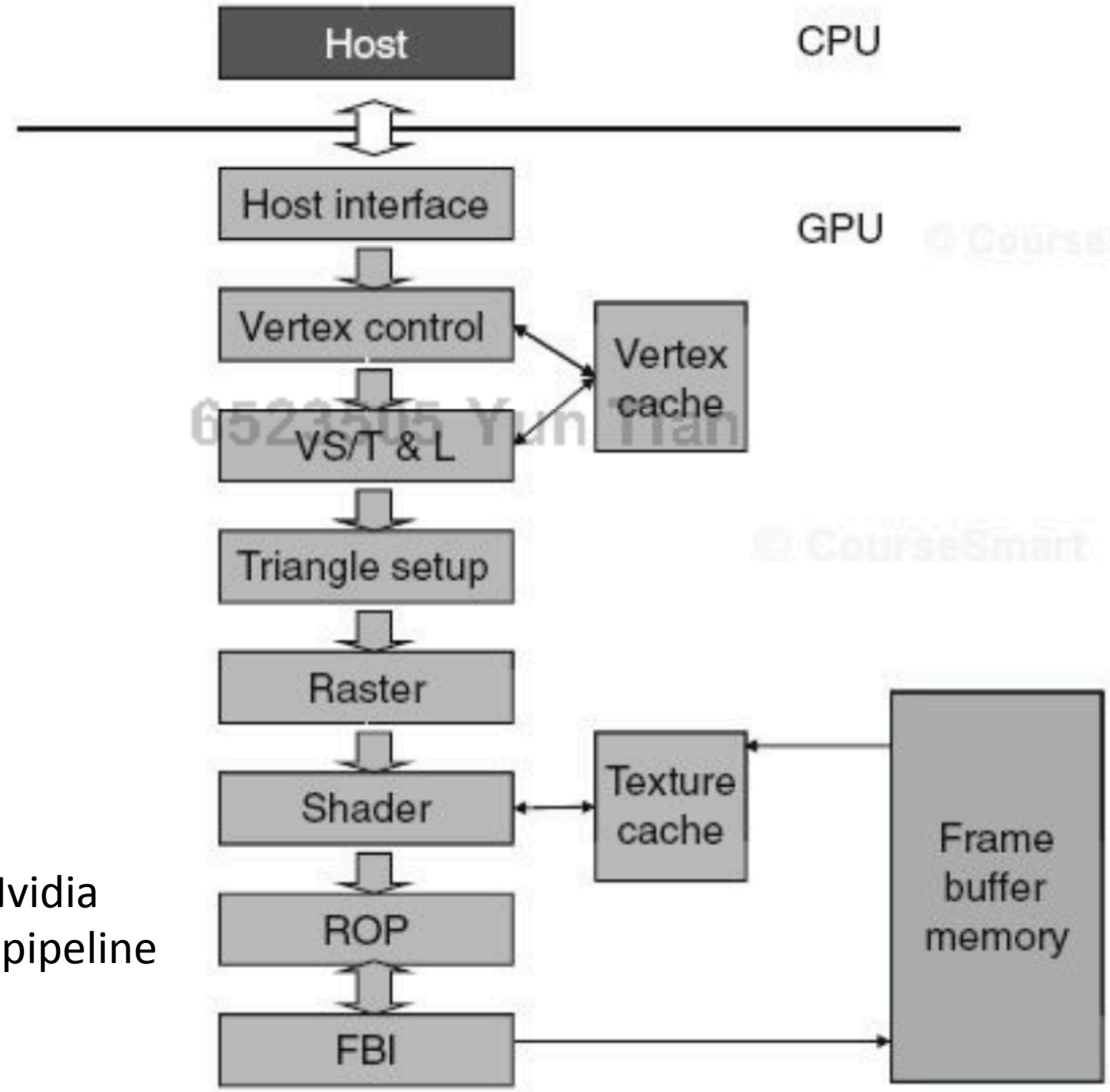
Old Fixed-Function Graphics Pipelines

- 1980s to 1990s, graphics hardware was fixed-function pipeline,
  - What is pipeline? (any idea?)
  - Each stage is configurable, but not programmable
- API for using GPU is available,
  - DirectX7 and below
  - OpenGL
  - E.g. Send a command to draw Object on a display

Old Fixed-Function Graphics Pipelines

- The GeForce graphics pipeline is designed to render triangles,
  - so the term **vertex** is typically used in this case to refer to the corners of a triangle.
- The surface of an object is drawn as a collection of triangles.
- The finer the sizes of the triangles are, the better the quality of the picture typically becomes.

Old Fixed-Function Graphics Pipelines



A fixed-function Nvidia GeForce graphics pipeline

Old Fixed-Function Graphics Pipelines

- Vertex Control Stage
  - Convert triangle data into a form that hardware understands, place them into vertex cache.
- VS/T&L
  - The vertex shading, transform, and lighting (VS/ T& L) stage,
  - Transforms vertices and assigns per-vertex values(e. g., colors, normals, texture coordinates, tangents).
  - but color is not applied to inside-triangle pixels until later.

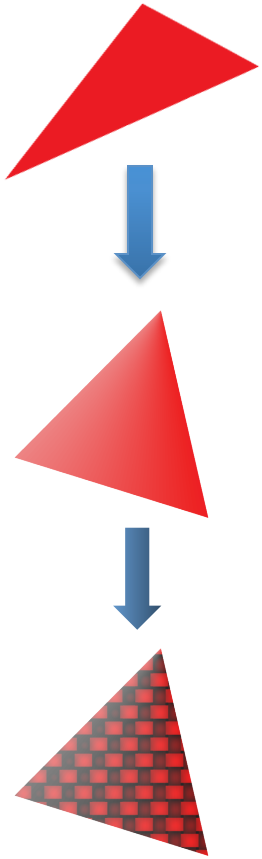
Old Fixed-Function Graphics Pipelines

- Triangle Setup
  - The triangle setup stage further creates edge equations that are used to interpolate colors and other per-vertex data.
- Raster
  - The raster stage determines which pixels are contained in each triangle and interpolates values used for shading.

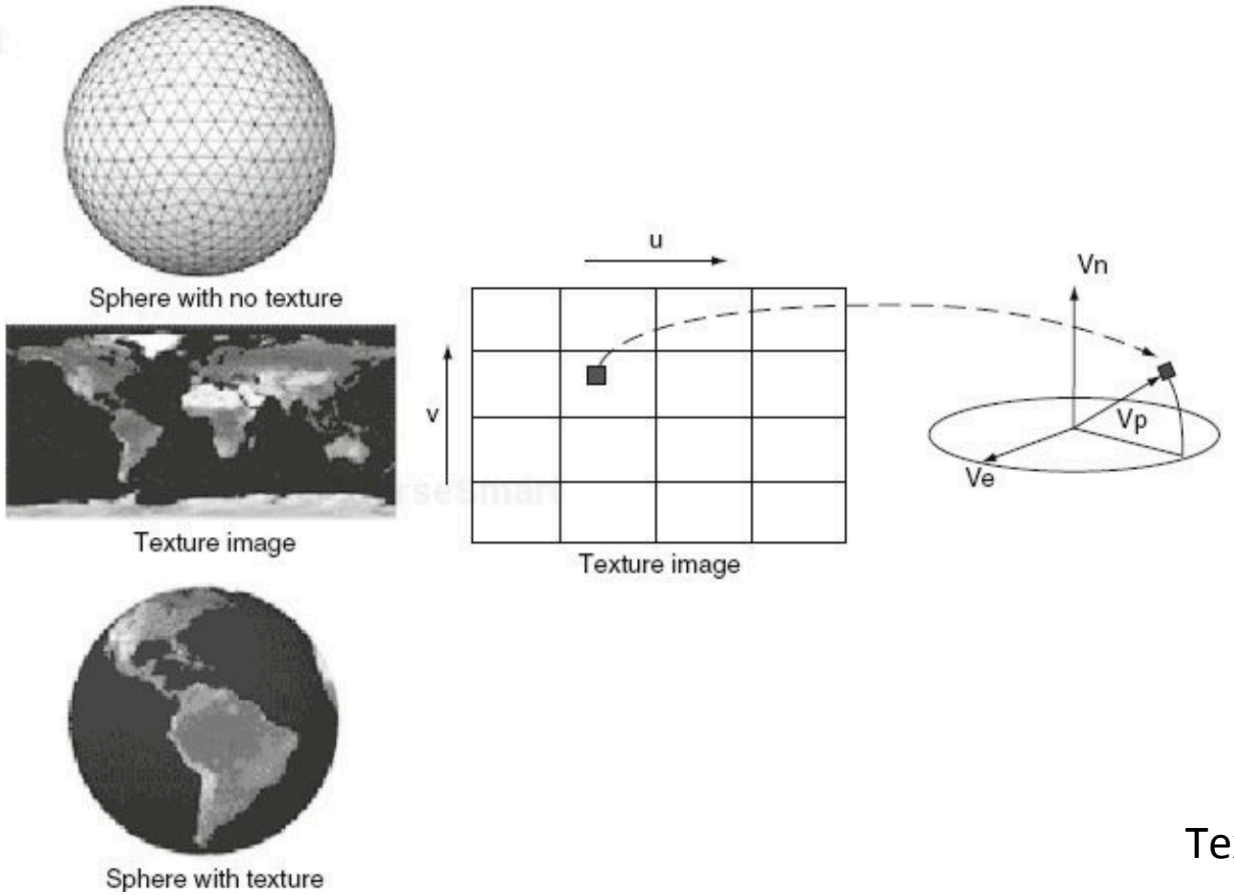
Old Fixed-Function Graphics Pipelines

- Shader
  - The shader stage determines the final color of each pixel.
  - This can be generated as a combined effect of many techniques:
    - Interpolation of vertex colors, texture mapping, per-pixel lighting mathematics, reflections, and more.
- ROP
  - Overlapping/adjacent color blending, transparency, anti-aliasing, discard occluded pixels

Old Fixed-Function Graphics Pipelines

- 

Texture Mapping

- 

Texture Mapping

Old Fixed-Function Graphics Pipelines

- Summary of Old Generation of GPUs
  - One chip/board per stage
  - Everything is fixed function, with a certain number of modes.
    - Number of modes for each stage grew over time.
  - Hard to optimize HW.
  - Developers always wanted more flexibility.
    - Especially during shading stage

Generation Two

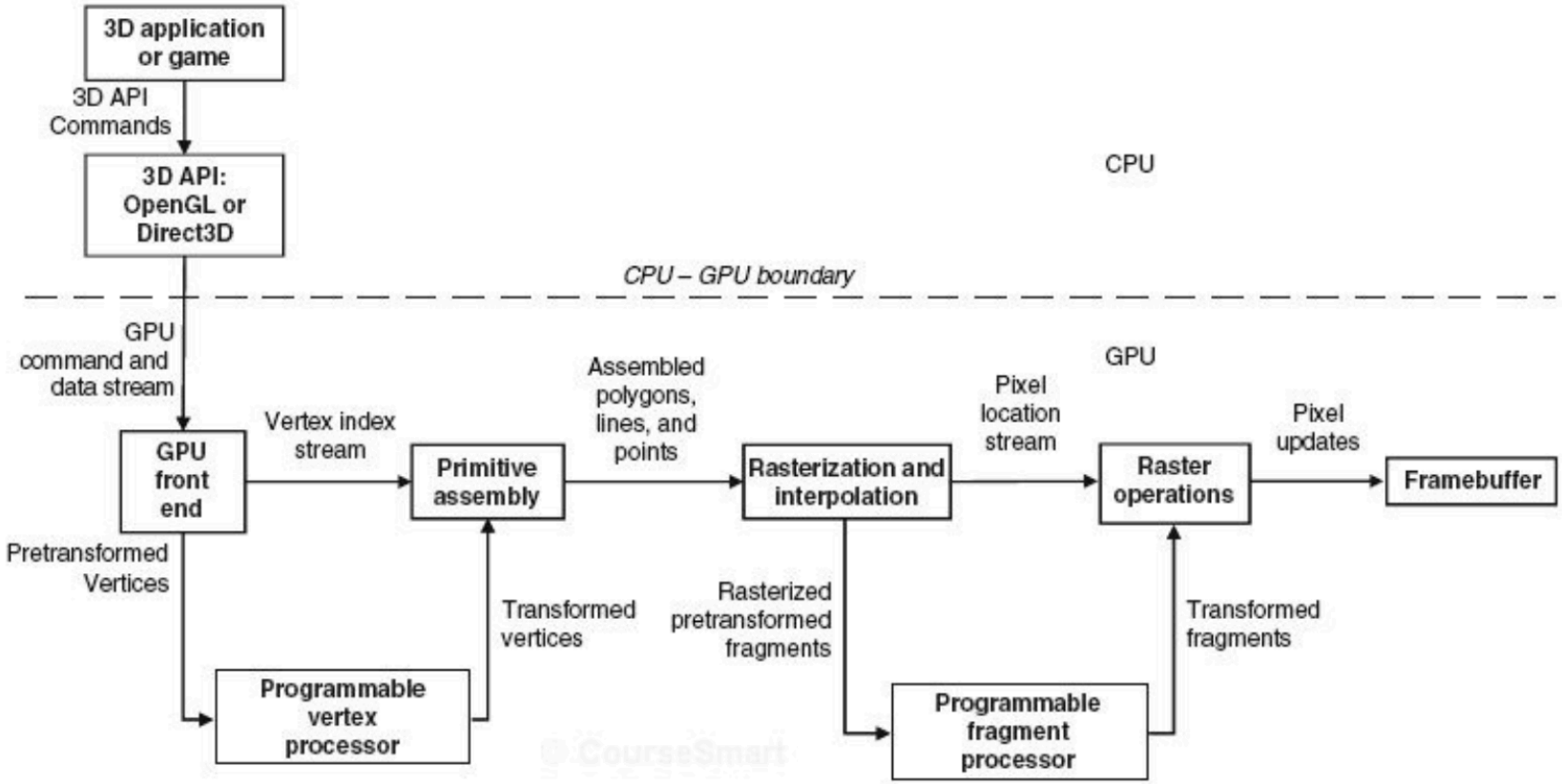


FIGURE 2.4

Example of a separate vertex processor and fragment processor in a programmable graphics pipeline.

Unified Graphics and Computing Processors

- The GeForce 8800 GPU mapped the separate programmable graphics stages to an **array** of unified processors;
- The logical graphics pipeline is physically a recirculating path that visits these processors three times,
  - with much fixed-function graphics logic between visits.

Unified Graphics and Computing Processors

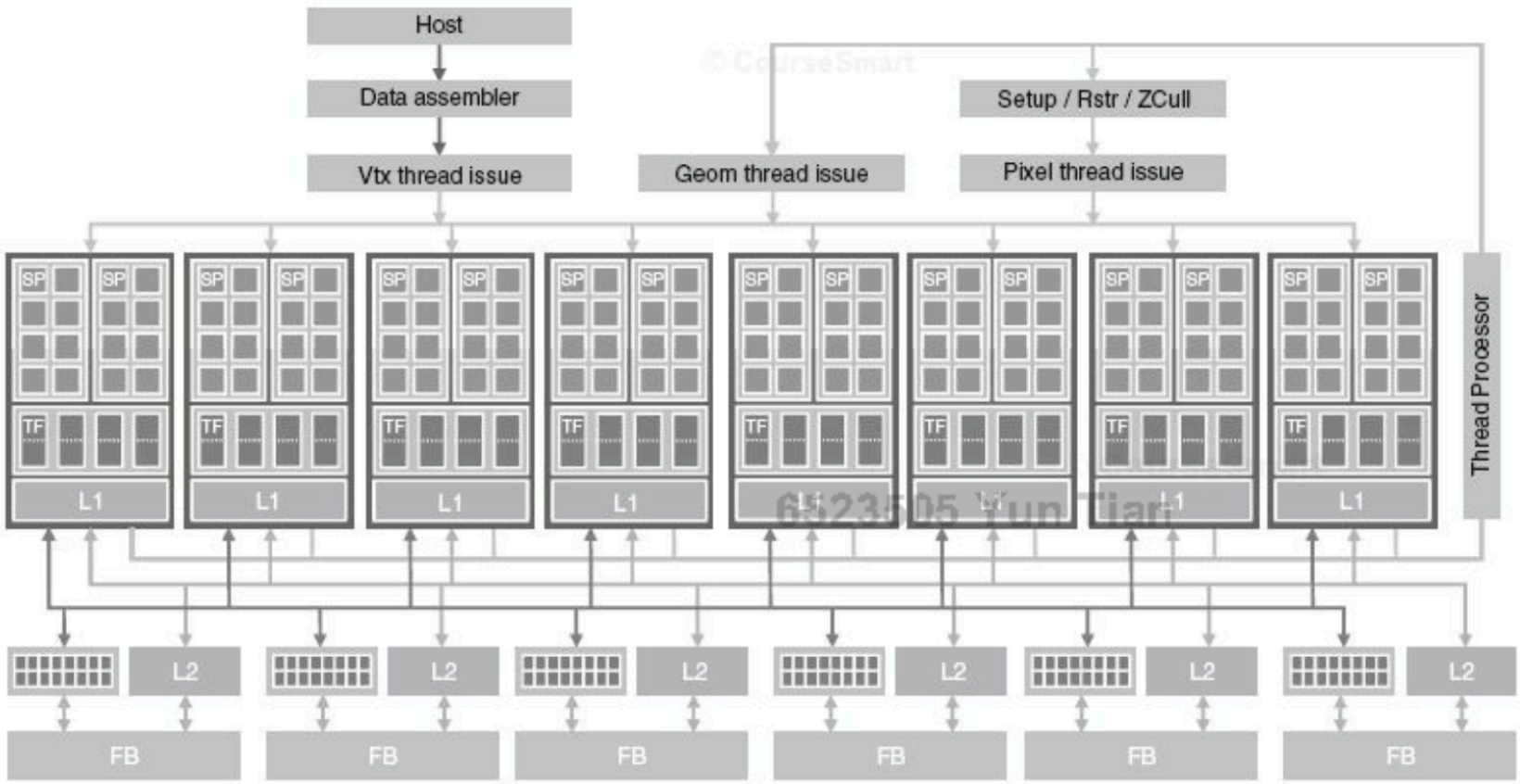


FIGURE 2.5

Unified programmable processor array of the GeForce 8800 GTX graphics pipeline.

Unified Graphics and Computing Processors

- The unified processor array allows dynamic partitioning of the array to vertex shading, geometry processing, and pixel processing.
- This unification allows the same pool of execution resources to be dynamically allocated to different pipeline stages and achieve better load balance.

Unified Graphics and Computing Processors

- Such Design paved the way for using programmable GPU processor array for general numeric computing.
  - Early GPGPU computing use the OpenGL API to do scientific computing.
    - Treat numeric data as triangles and vertices
    - Use GPU for other purpose.
    - A lot of pain

Scalable GPUs

- Workload and Programming Model provide lots of parallelism.
- Applications provide large groups of vertices at once
  - Vertices can be processed in parallel
  - Apply same transform to all vertices
- Triangles contain many pixels
  - Pixels from a triangle can be processed in parallel
  - Apply same shader to all pixels

Scalable GPUs

- Very efficient hardware to hide serialization bottlenecks.
  - Big latency for one I/O operation
  - But single I/O grabs a lot data. (high bandwidth)
  - Thousands of threads active maximize throughput and hides latency.
  - Thread creating and switching almost free.

Auto Scale Compared with CPU

- CPUs scales by increasing the number cores rather than simply increasing the performance of a single core.
  - Quad-core, hex-core and oct-core CPUs emerge.
- CPU programmers usually use coarse-grain parallelism.
  - Different task of an application performed in parallel.
  - E.g. an array is divided into to be processed on 4 cores.

Auto Scale Compared with CPU

- Such CPU applications must be rewritten often
  - when executed on a machine where number of processor cores are doubled.

Auto Scale Compared with CPU

- In contrast, GPUs uses massive, fine-grained data parallelism.
  - Efficient threading support in GPUs (use hardware)
  - Applications expose a much larger amount of parallelism(thousand threads) than available hardware execution resources with little or no penalty.
  - In comparison, threading on CPU is more expensive. (create, switch),
    - Impractical to create such large number of threads.

Auto Scale Compared with CPU

- Once in program one created thousands of threads,
- each doubling of number of GPU cores provides more hardware execution resources,
  - Supports and exploit more of the exposed parallelism for higher performance,
- Parallel computing is designed for transparent and portable scalability.

Auto Scale Compared with CPU

- A graphics program or CUDA program is written once,
- It runs on a GPU with any number of processor cores.
  - Existing CUDA programs gets higher performance without any change when executed on newer and more powerful GPUs.

Summary

- Overview of Graphics
- Graphics Pipeline and its Evolvement
- Programmable Array of Unified GPUs processors.
- Auto Scalable GPUs.(transparent scalability)
- Please read textbook chapter 2.

Next Class

- Introduce CUDA
- Simple CUDA Program